Home Setup: STM32CubeMX + VS Code

EE5203 Microprocessors — set up your own computer in seven steps

Basri Erdogan — 2026-2027 Fall

With these tools you can do labs and homework on your own computer. Plan about **one hour**. Most of it is waiting for downloads.

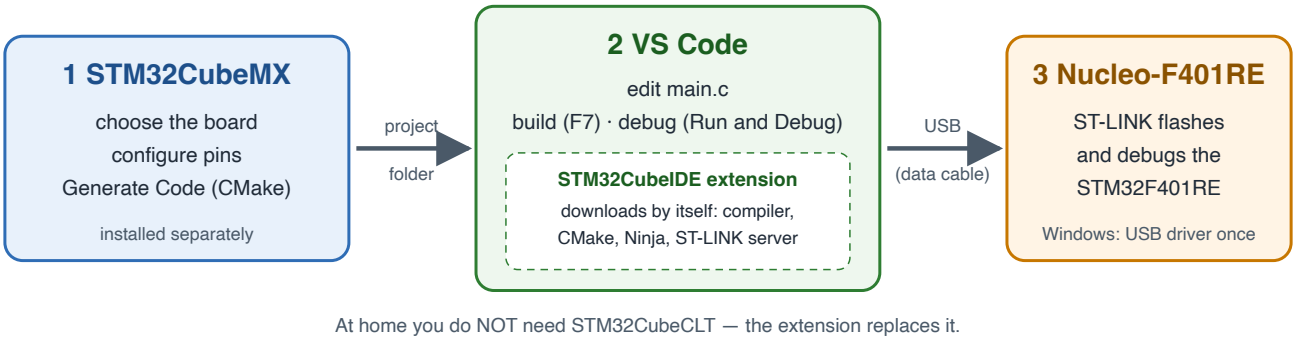


Figure 1: CubeMX makes the project, VS Code builds it, the board runs it.

> **i** No board yet? No problem.
>
> Do **steps 1–6**. They end with a successful build, and that is enough for now. When you have your board, do **step 7** to run and debug the program.

Before you start

- You need a Windows 10/11 (64-bit) or macOS computer, **10 GB** free disk, and an internet connection.
- Create a free **myST account** at st.com (top right, *myST*). You need it to download CubeMX.
- Keep your projects in `C:\STM32\EE5203` (macOS: `~/STM32/EE5203`). Never use a folder with **spaces** or **Turkish characters** (ç ğ ı İ ö ş ü).

1. Install VS Code

Download it from code.visualstudio.com and install it with the default options.

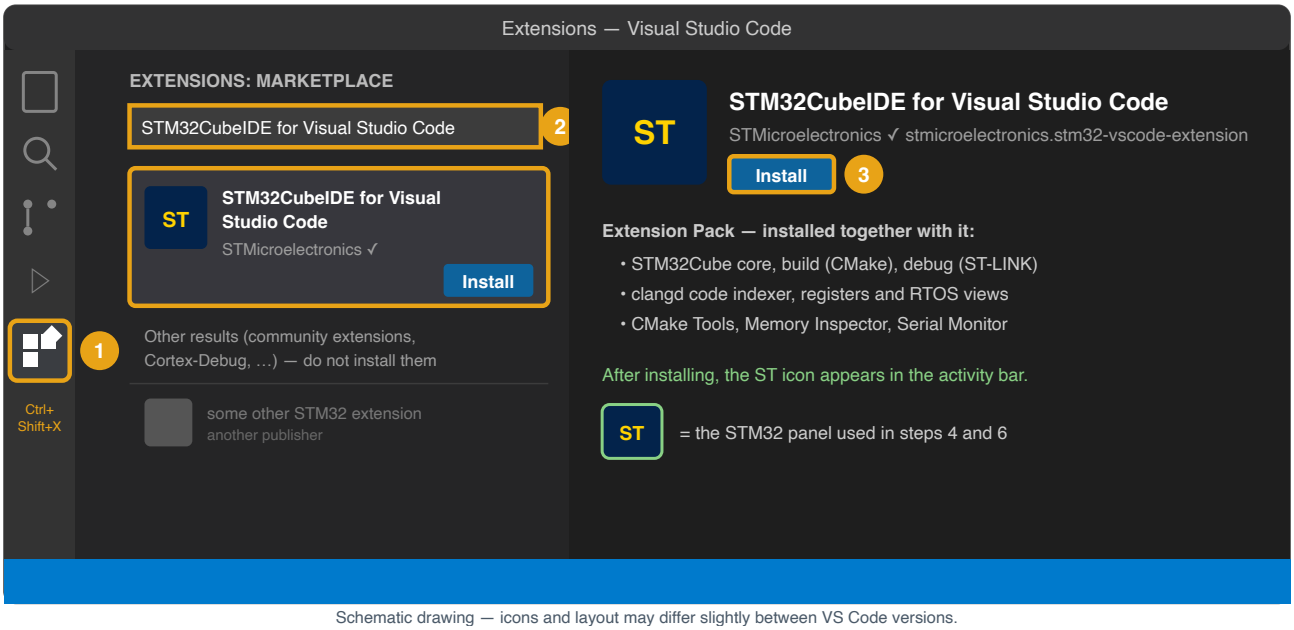
2. Install STM32CubeMX

Go to st.com → STM32CubeMX → *Get Software*, log in, and download the package for your computer:

- **Windows:** *STM32CubeMX-Win*. Unzip it and run `SetupSTM32CubeMX-6.18.1.exe`.
- **Mac with Apple chip (M1–M4):** the **Arm64** package.
- **Mac with Intel chip:** the macOS package.

Accept the defaults. You do not need to install Java.

3. Install the STM32 extension



Schematic drawing — icons and layout may differ slightly between VS Code versions.

Figure 2: (1) Extensions, (2) search, (3) Install the one from STMicroelectronics.

1. In VS Code, open **Extensions** (Ctrl+Shift+X).
2. Search **STM32CubeIDE for Visual Studio Code**.
3. Click **Install** on the one by **STMicroelectronics**.

The **ST** icon appears on the left. This extension installs its helpers and downloads the compiler later. Do not add Cortex-Debug or other STM32 extensions.

⚠ Windows: does your user name have a space or a Turkish character? (click)

Open **Command Prompt** and type `echo %USERPROFILE%`. If you see something like `C:\Users\Ayşe Yılmaz`, do this once:

1. Create the folders `C:\ST\bundles` and `C:\ST\packs`.
2. Start → type **environment** → *Edit environment variables for your account.*
3. Add two variables: `CUBE_BUNDLE_PATH` = `C:\ST\bundles` and `CMSIS_PACK_ROOT` = `C:\ST\packs`.
4. Close and reopen VS Code.

ℹ Mac: Terminal setup and shortcuts (click)

1. In **Terminal**, run `brew --version`. If it is missing, install Homebrew and follow its **Next steps** to set up your PATH.
2. Run `brew install zstd`, then restart VS Code.

For the Ctrl shortcuts below, use **Cmd** on Mac. F7/F5 may need **Fn**.

4. Create the project in CubeMX

1. **File → New Project → Board Selector** → type **NUCLEO-F401RE** → select it → **Start Project**.
2. *Initialize all peripherals with their default mode?* → **Yes**.
3. If CubeMX asks to download the **STM32CubeF4** package → **Yes** (large download, one time only).
4. **Project Manager:** Name = `EE5203_Home_Blink`, Location = `C:\STM32\EE5203` (Mac: `~/STM32/EE5203`), Toolchain = **CMake**. Click **GENERATE CODE**.

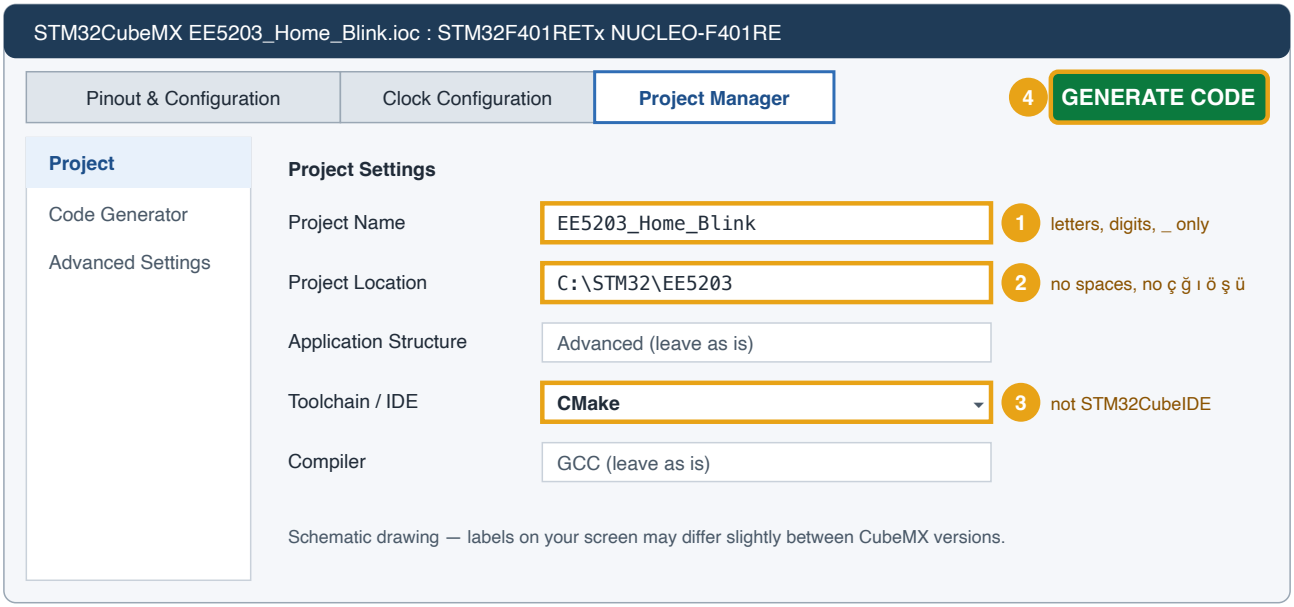


Figure 3: (1) Name, (2) Location, (3) Toolchain = CMake, (4) Generate Code.

5. When CubeMX asks *Open Project?*, click **Close**.

5. Open the project in VS Code

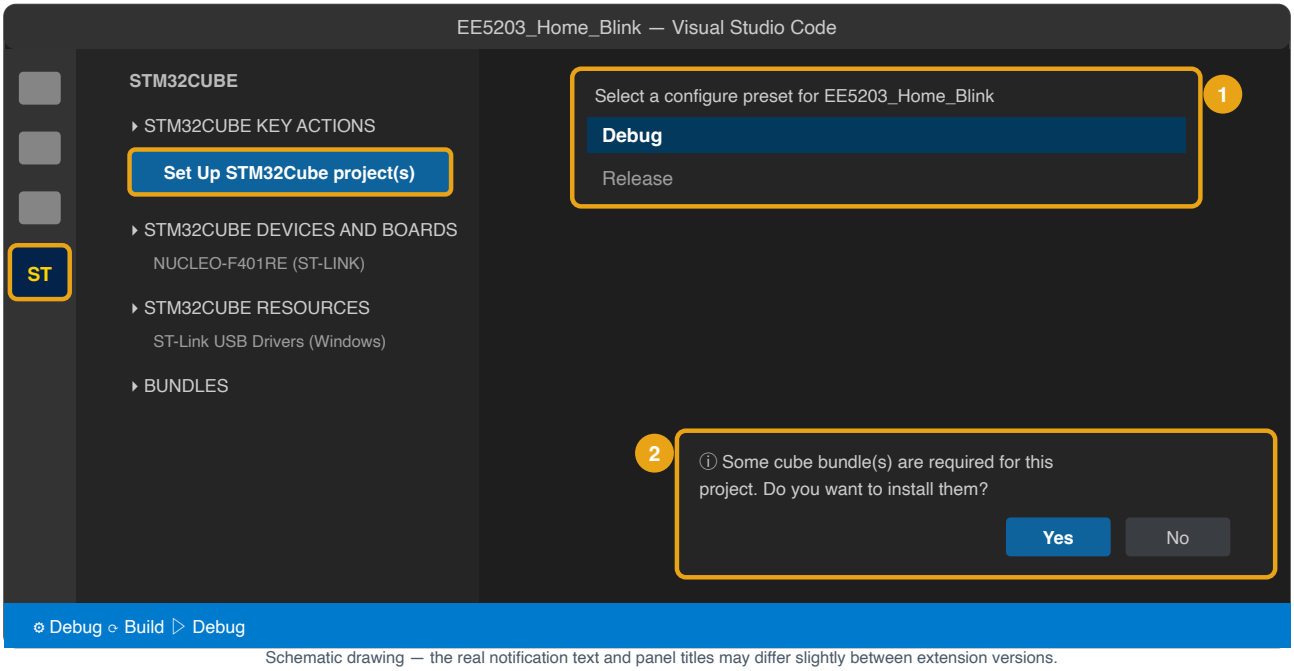


Figure 4: (1) Debug, (2) Yes, (3) Set Up STM32Cube project(s).

1. **File → Open Folder...** → select `EE5203_Home_Blink` (the folder with the `.ioc` file).
2. **(1)** Choose the preset **Debug**.
3. **(2)** *Install cube bundle(s)?* → **Yes**. The first download takes a few minutes.
4. **(3)** ST icon → **Set Up STM32Cube project(s)**. If prompted, select **NUCLEO-F401RE** / **STM32F401RE**, **GCC**, and the **main project** → **Save and close**.

6. Add the code and build

In `Core/Src/main.c`, add this between `USER CODE BEGIN 2` and `USER CODE END 2`:

```
int toggle_count = 0;
```

Inside `USER CODE BEGIN 3`, before the existing `}`, add:

```
HAL_GPIO_TogglePin(LD2_GPIO_Port, LD2_Pin);
toggle_count = toggle_count + 1;
HAL_Delay(500);
```

Save (Ctrl+S) and press **F7** to build.

Success: `Linking C executable EE5203_Home_Blink.elf`, followed by `Build finished with exit code 0`.

> ! Build error `...FLASH.ld: syntax error`?
>
> This is a known CubeMX bug. Download STM32F401RETx_FLASH.ld, copy it into your project folder to **replace** the old one, and press F7 again. (Repeat this after every *Generate Code*.)

No board yet? You are done — your computer is ready.

7. With a board: run and debug

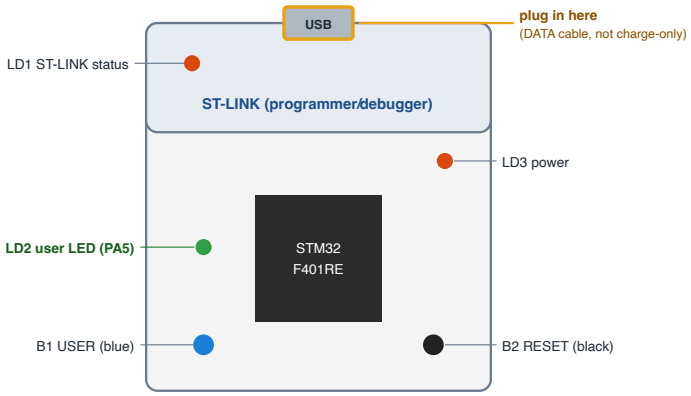


Figure 5: Plug the cable into the USB connector at the top (ST-LINK side). LD2 is the LED that will blink.

1. **Windows only, once:** ST icon → **STM32CUBE RESOURCES** → **ST-Link USB Drivers** → install.
2. Plug in the board with a **data** cable. If VS Code offers an **ST-LINK firmware update**, accept it.
3. Open **Run and Debug** (Ctrl+Shift+D) → **Run and Debug** → choose **STM32Cube: STM32 Launch ST-Link GDB Server**.
4. The program stops at `main`. Press **Continue** (F5): **LD2 blinks.**
5. Click left of `toggle_count = toggle_count + 1;` to set a **breakpoint** (red dot). Each Continue stops there again; watch `toggle_count` grow in **VARIABLES**.

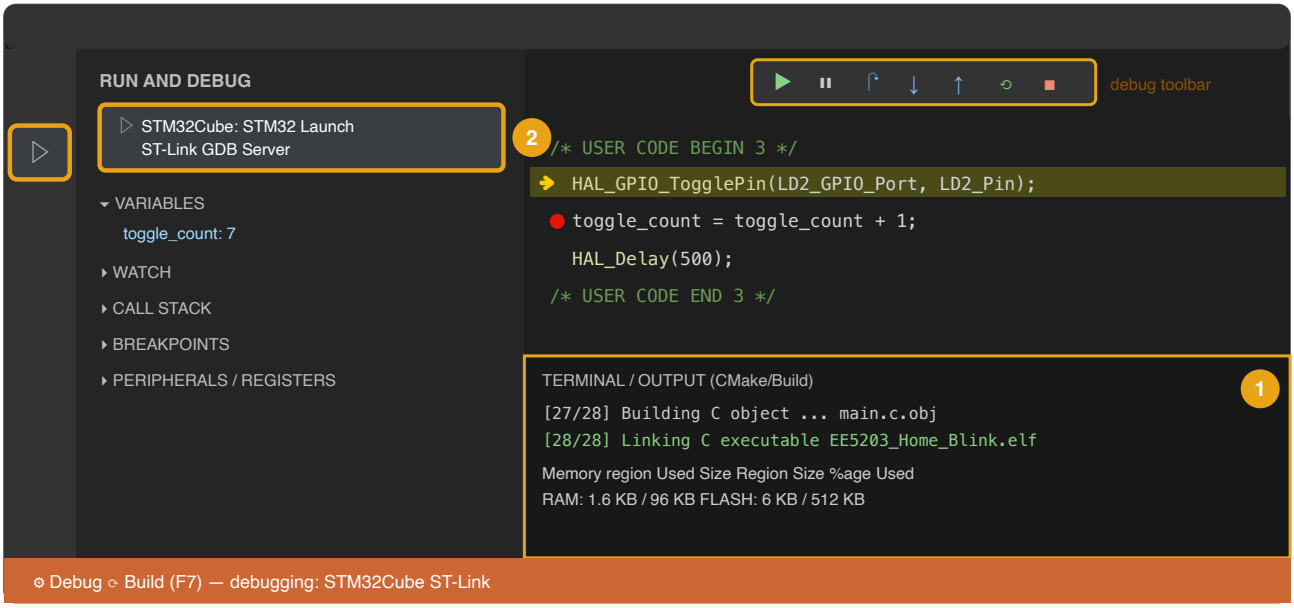


Figure 6: The debug configuration to choose (2), next to the build output (1).

If something goes wrong

- **Board not found:** try another USB cable (many are charge-only), install the driver (step 7.1), close CubeMX, unplug and replug.
- **Red lines under `#include` but the build works:** see your Windows or Mac box in step 3.
- **Bundle download never finishes:** check your internet; some dorm or company networks block it — try another network.
- **My code disappeared after *Generate Code*:** it was outside the `USER CODE` comments. Always write inside them.

Still stuck? Send your lab assistant the **step number** and the **first error line** (as text, not a photo), or bring your laptop to the lab.

> **i** Lab PC or home? (click)
>
> At home use **F7** and the **STM32Cube** debug configuration; the lab uses **Ctrl+Shift+B** and **F5**. No separate CubeCLT install is needed at home. To bring a lab project home, copy its folder to your EE5203 folder, remove `build`, rename `.vscode` to `.vscode_lab_backup`, then follow step 5.